

Trees (Basic Operations)

(Insertion)

Node* insertion(Node* root, int val)

```
if (root == NULL)
```

// if 1st Node

```
{
```

```
    return new Node(val);
```

// create & return

```
}
```

```
if (val < root->data)
```

// suppose we already inserted 20 & Now 10

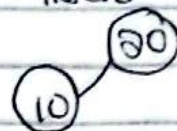
```
{
```

```
    root->L = insertion(root->L, val);
```

So mov to L as 10 < 20

```
}
```

& insert:



```
if (val > root->data)
```

// Now, if we insert

```
{
```

30, as 30 > 20 so

```
    root->R = insertion(root->R, val);
```

mov to R & insert

```
}
```

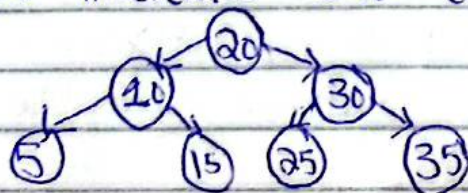


// mov to L or R untill reaches null at L/R.

Return root;

(Suppose)

After few insertions our tree is:



(Deletion)

```
Node* deletenode(Node* root, int key)
```

```
{  
    if (root == NULL)
```

```
    return;
```

```
}  
// if root = NULL, means node not  
found.
```

```
if (key < root->data)
```

```
{  
    root->L =  
    deletenode(root->L, key);
```

①

Sup: we want to delete 15

root->d = 20, 15 < 20

So the above function will be called

then,

root->d = 10

```
if (key > root->data)
```

```
{  
    root->R =  
    deletenode(root->R, key);
```

②

Now root->d = 15, 15 > 10

So this function will be called

then, root->d = 15

else if (k == x->d) ✓ 15 = 15

```
{  
  if (root->L == NULL) ✓  
  {  
    Node* temp = x->R;  
    delete root;  
    return temp;  
  }
```

// as root->d = key
// and root->L = NULL, so we will store
it's right into temp & delete it &
then return temp to its caller. that
was (b) &
& then after completion it will
also return to (a)

// ROC Now:

```
else if (x->R == N)  
{  
  Node* t = x->R;  
  delete root;  
  return temp;  
}
```

This also
satisfies as
15->R is also
Null.

```
else  
{
```

// if 15's L/R = N then
find min from R subtree

```
Node* temp = findmin(x->R); // & store in temp.
```

```
x->d = t->d; // then put its d into x & then
```

```
x->R = deletenode(x->R, t->d); // call fn to
```

// del that T->d from

```
}  
return root;
```

x-R;

(Find Min)

```
Node* findmin(Node* root)
```

// if not found

```
{  
    if (root == NULL)  
    {  
        cout << "No";  
    }
```

```
    while (root->L != NULL)
```

// move to most left
recursively until

```
{  
    root = root->L;  
}
```

```
return root;
```

```
}
```

(Inorder Traversal)

```
void inorder(Node* root)
```

// if root == NULL

```
{  
    if (root == NULL)
```

```
{  
        return;
```

```
}
```

```
else
```

```
{  
    inorder(root->L);
```

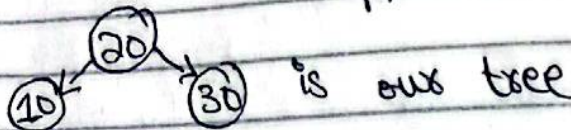
```
    cout << root->data;
```

```
    inorder(root->R);
```

```
}
```

```
}
```

(Suppose)



Node 20 → root

1) root->L = 10

again 1) root->L = NULL return

cout $\rightarrow$ 10
 6 $\rightarrow$ R $\rightarrow$ 20
 6 $\rightarrow$ R $\rightarrow$ 30
 6 $\rightarrow$ R $\rightarrow$ N ref
 6 $\rightarrow$ R $\rightarrow$ 20 ref

cout $\rightarrow$ 20

6 $\rightarrow$ R $\Rightarrow$ 30
 6 $\rightarrow$ L $\Rightarrow$ N ref
 cout 30

(Preorder Traversal)

void po(N* r)

{
 if (-) { return; }
 else

{
 cout << -;
 po(r->L);
 po(r->R);
 }

cout first
 then move to
 L & then R

(Postorder Traversal)

void pe(N* r)

{
 if (-) { return; }
 else {

po(r->L);
 po(r->R);
 cout << -; }

move to L first
 then R & then
 cout;

(Search_Node)

```
bool searchnode(Node* root, int key)
{
    if (root == NULL) { return false; } // if not found
    if (key == root->data) { return true; } // if found
    else if (key < root->data) {
        // move recursively to left until k < root->data
        return searchnode(root->left, key);
    }
    else {
        // move recursively to right until k > root->data
        return searchnode(root->right, key);
    }
}
```

Suppose:



This is our tree

root = 20, k = 10

(a)

if (root == NULL) ✗
if (key == root->data) ✗

else if (key < root->data) ✓
{

return searchnode(root->left, key);

(b)

if (—) ✗

if (—) ✓

10 = 10, set true

(Update_Node)

```
void updatenode(int ov, int nv)
{
    if (searchnode(root, ov)) // if the val is found
    {
        root = deletenode(root, ov); // del that node
        root = insertnode(root, nv); // insert new value/
        // node
    }
    else { "NO"; } // if not found
}
```

(Find_Node)

```
Node* findnode(Node* node, int key)
{
    if (node == NULL) { return NULL; } // if not found
    else if (n->d == key) { return node; } // if found
    else if (k < n->d)
    {
        return findnode(node->L, k);
    }
    // if k < n->d move up to L until this
    // satisfies
    else if (k > n->d)
    {
        return findnode(node->R, k);
    }
    // mov recursively to R until this satisfies
}
```

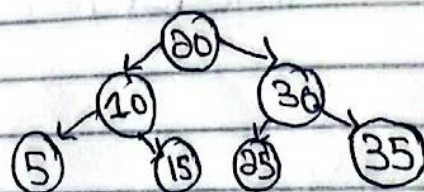
(Find-Parent)

```

Node* findparent (Node* par, int val)
{
    if (par == NULL) // if not found
    {
        return par;
    }
    else if ((par->L && par->L->d == val) ||
             (par->R && par->R->d == val))
    {
        return par;
    }
    else if (val < par->d)
    {
        return findpar(par->L, val);
    }
    else if (val > par->d)
    {
        return findpar(par->R, val);
    }
}

```

→ Suppose this is our tree



par → 20
val = 5

if (par == N) ✗
else if (—) ✗

✓
((20 → L ≠ N && 20 → 10 ≠ 5) || (20 → R ≠ N && 20 → 30 ≠ 5))

else if (val < p->d) ✓, 5 < 10

② return fp(→) ✓;

→ fp(→)

pa → 10, val = 5

if(→) α

else if(→) ✓

(par → L ≠ N) && (par → R = 1) ✓

return par; to ①

return 10; to caller

Count_Nodes

int CN(Node* root)

if (root == NULL) { return 0; }

// return 0 if = N

{

return 1 + CN(x → L) + CN(x → R);

// recursively
call fn &
add 1 each
time

}

}

Node_height

int nh(int val)

Node* node = findnode(root, val); // find node first

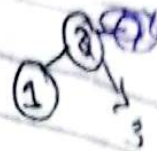
if (node == NULL) { return -1; } // if not found

else { return height(node); } // call height fn for that node.

BST_TREE (ALL - Questions)

(Question # 01)

findheight(Node* root)



if (x == N) { return 0; } x

x = 2

int lh = fh(—);

①) fh(—) x x = 1

if (—) x

int lh = (—);

②) fh(—)

if (—) ✓

x = N

{ return 0; } to ①

lh = 0

int rh = (—)

③) fh(—)

if (—) ✓

x = N

{ return 0; }

to ②

rh = 0

return 1 + max(lh, rh) → return 1 + 0 = 1

to ①

lh = 1

// same for next rh recursing

rh = 1

return 1 + max(1, 1) 1 + 1 = 2 to main;

(Question # 02)

① int countLeaves(node)

$n = 3$

if ($n == N$) α

if ($n \rightarrow L == N$ && $n \rightarrow R == N$) α

return cntL($n \rightarrow L$) +

$n = 1$

② cntL($—$)
if ($—$) α
if ($—$) $\checkmark$
{
return 1;
}

return 1 + cnt($n \rightarrow R$)

③ cnt($—$) $n = 5$
if ($—$) α
if ($—$) $\checkmark$
return 1

return 2;

(2 leaf nodes)

int cntL($—$)

$n = 3$

if ($—$) α

return 1 + (0) + cnt($n \rightarrow R$)

if ($—$) $\checkmark$

return 1 + cnt($n \rightarrow L$)

④ cntL($—$)

if ($—$) α

if ($—$) α

return 0

⑤ cntL($—$)

if ($—$) α

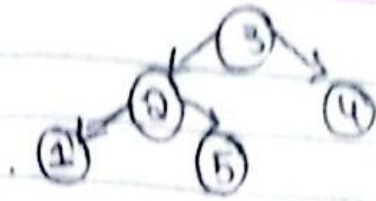
if ($—$) α

return 0

return 1 + 0 = 1

(1 non-leaf)

(S#00)4



3 2 4 1 5

enqueue (root):

ary[1] = 3

Node* aux = q.dequeue();

set 3

cout << c->d;

3

if (c->L != N) ✓ 3->L = 2

{ enqueue(c->L);

ary[2] = 2

if (c->R != N) ✓ 3->R = 4

{ enqueue(c->R);

ary[3] = 4

while (!q.isempty())

Node* aux = q.dequeue(); set 2

cout << c->d;

2

if (c->L != N) ✓ 2->L = 1

{ enqueue(c->L);

ary[4] = 1

if (c->R != N) ✓ 2->R = 5

{

enqueue(c->R);

ary[5] = 5

}

2) ag while(!q.isrep()) ✓
{

Node* cur = q->deg();

cout << cur->d;

if (cur->L != N) α

if (cur->R != N) α

ary = {4, 1, 5}

set 4

4

4 → L = N

4 → R = N

2) ag while(—) ✓
{

Node* cur = q->deg();

cout << cur->d;

if (—) α

if (—) α

ary {1, 5}

set 1: ary → {5}

1

2) ag while(—) ✓
{

Node* cur = q->d();

cout << cur->d;

if (—) α

if (—) α

set 5 → ary {N}

5

2) ag while(—) α

(Q# 05)

diameter = 6



int HAD(Node* t, int &ed) n=1, d=0

if (t == NULL)

return 0;

⑤ had(t) n=2, d=0

{

if (t == NULL)

return 0;

⑥ had(t) n=4, d=0

{

if (t == NULL)

return 0;

⑦ had(t) n=N, d=0

if (t == NULL)

return 0; to ③

int Lh=0;

int Rh=had(t->r);

⑧ had(t) n=N, d=0

if (t == NULL)

return 0; to ③

int Rh=0;

int ed=0

if (t == NULL)

return 1+max(Lh, Rh) = 1

return 1; to ③

int Lh=1;
int Rh=1;

② had(-) n=5, d=0
{

if(-)X
int Lh=1;

④ had(-) n=N, d=0

if(-)✓

set 0; to c

int Lh=0;

int Rh(-);

③ ha(-) n=N, d=0

if(-)0

ret 0; to c

int Rh=0

if(-)X

ret 1+max(0,d); ret 1 to ①

int Rh=1;

in cd=1+1=2

if(-)✓

{ d=cd; } (d=2)

(Question # 06)

Suppose



bool isbst(—)

```

if (node == NULL) &
if (n->d == —) &
return isbst(n->L, —)
  
```

n = 3

⑥ isbst(—)

if (n == —) &

if (—) &

return isbst(n->L, —)

n = ②

⑦ isbst(—)

if (—) return true; to b

return true && isbst(n->R)

⑧ isbst(—)

if (—) return true to ⑥

return true; // to ⑦

return true && isbst(—);

⑨ isbst(—)

if (n == —) &

if (—) ✓

return false; // to ⑧

return true && false → false

n = 1

1 < minval (1 < 3)

could not a bst;

~~Question # 07~~

Q # 07

path[0] = 3
pathLen++;



if (node->d == 2) {

t = 1

if (fp(n->l, -) ||

o) path[2] = 2
PL++;

path[0] = 3
PL = 0;
path[0] = 3

if (-) {

if (pp(-) ||

path[0] = 2

path[2] = 1

o) path[2] = 1
PL++;
if (-) {
true

path[1] = 2
path[2] = 5

set true

(Question # 08)

2nd phase



int

sumofleftleaves(N, bool isL)

n = 5

if (n == N) {

if (n == L2) {

int LS = SL(n -> L, true);

n = 2

SL ->

{

if (n == N) {

if (L ->) {

int Lst = SL(n -> L, T);

SL ->

n = 1

if (n ->) {

if (L ->) {

ret n -> d;

LS = 1;

int RS = SL(n -> R, F);

SL ->

n = N

if (L ->) {

ret 0;

RS = 0;

ret LS + RS = 1; to a

LS = 1

int RS = SL -> ;

// same recursive

Q # 09



if (patn == ...)

(a) node = 3, tat = 5

if (—) x

if (—) x

if (—) ||

→ 3

enter

(5) ✓

(b) node = 3, tat = 5

if (—) x

if (—) x

if (—) ||

→ (5)

ret

(c) node = 1, tat = 5

if (—) x

if (—) ✓

set tru

~~Q # 9~~ Q # 10

(1) node = 3, k = 3rd, c = 0, b = -1

if (—) x

(2) (3)

if (—) x

(3) (1)

if (—) x

↓ (4) — set

c = 1;

(Question # 18)



void inorder(N* n, int &c, int k, int &ret)

n=3
c=0
k=3
c=1

if (n==N) {
inorder(n->L, c, k, ret);

⑥ inorder(—)

n=1

if (L==N) {

in(—)

③ in(—)

n=N

if (—)

ret

cout<<H;

c=1

if (c==k) {

in(n->R);

② ino(—)

n=N

if (—)

ret;

c++;

if (—) {

ret = n->val;

ret;

}

kth smallest node is 8
2nd smallest node is 13

AVL - TREE (ALL - Questions)

(Question # 01)

// Remaining insertion is same as BST

Node* insertion(Node* root, int k, bool & dup)

```
{
    if (n == N)
    {
        return new Node(k);
    }
    if (L == -1)
    {
        Same;
    }
    if (K > -1);
    {
        Same;
    }
    else
    {
        dup = F;
        return node;
    }
}
```

Same as BST

updateHeight(node);

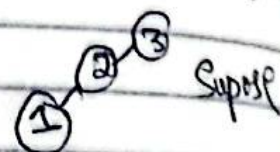
// update height

int bal = get(node)

// to check for

Case 1: LL:

```
{
    return rr(node)
}
```



case 2: RR
if (—)
{
 return node;
}



case 3: RL:
if (—)
{
 node->L = L(n->L);
 return node;
}



case 4: RL:
if (—)
{
 node->R = R(n->R);
 return node;
}



(Question # 02)

// Remaining deletion is same as bst

⇒ New parts:

updateheight(node);

// update height for
that returned node

int bal = getbal(node);

// get its balance
factor

// Remaining part is same as of insertion's
new part of balancing.

(Question # 09)

```
void pbF(Node* node)
```

```
{  
    if(n==0)
```

n=5



```
    int lh = height(n->L);
```

lh = 2

```
    int rh = height(n->R);
```

rh = 2

```
    if(lh == rh)
```

bf = 0

```
        pbF(n->L);
```

rh = 2

```
        (b) pbF( )
```

```
            if( )
```

lh = 0

```
            int lh = ;
```

rh = 0

```
            int rh = ;
```

bf = 0

```
            int bf = ;
```

```
            pbF(n->L);
```

```
            (c) pbF( )
```

```
                if( )
```

```
                return;
```

cout << "PBF of 2 is 0"

```
            pbF(n->R);
```

```
            (d) pbF( )
```

```
                if( )
```

```
                return;
```

cout << "PBF of 5 is 0"

```
            pbF(n->R);
```

continue;

(Question # 04)

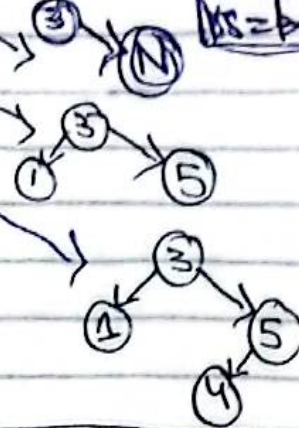
Rotate Right (Node* ubn) ubn=5

{ Node* ev = ubn->L; ev=3

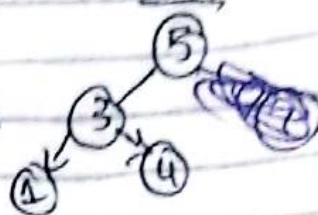
Node* xs = ev->R; xs=4

ev->R = ubn;

ubn->L = xs;



Suppose:



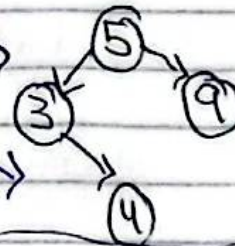
Rotate Left (Node* ubn)

{ Node* ev = ub->R; ev=5

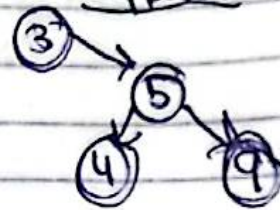
Node* xs = ev->L; xs=4

ev->L = ubn;

ubn->R = xs;



Suppose:



LR (Node* ubn)

{

ubn->R = RL(ubn->R); ubn=

rel RL(ubn);

}

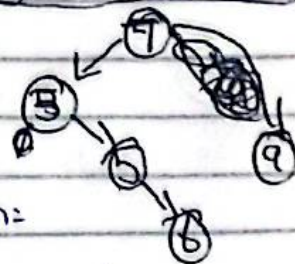
RL (Node* ubn)

{

ubn->L = RL(ubn->L);

rel RL(ubn);

}



Recursively
call
the
rotation
fun

(Question # 05)

bool is_balanced(Node* n)

if (n == NULL) {

n = 5

return true;

}

int lh = height(n->L);

lh = 2

int Rh = height(n->R);

Rh = 1

int bf = lh - Rh;

bf = 1

if (bf > 1)

return is_balanced(n->L) && is_balanced(n->R);

It's same as Q # 03, just a check added for balance factor.

(Question # 06)

Node* SAtoAVL(int arr[], int st, int end)

↓
a

int mid = (st + end) / 2; mid = 1

Suppose: arr is {1, 3, 5, 7}

Node* node = new Node(arr[mid]);

node->L = SAtoAVL(arr, st, mid-1);

3

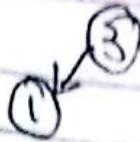
node->R = SAtoAVL(arr, mid+1, end);

b) 0) SAtoAVL(arr, st, mid-1);

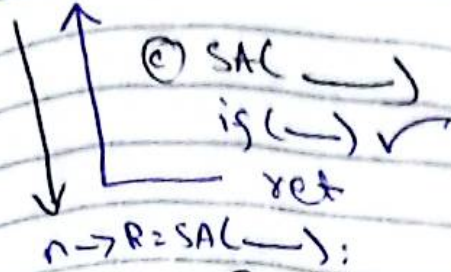
int ni = ...;

mid = 0;

nodz —;



$n \rightarrow L = \text{SAC}(\text{---});$



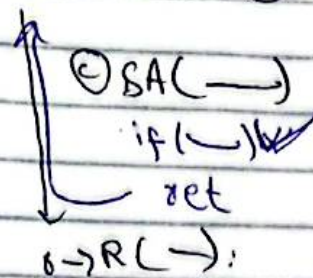
(a) SAC(---)
if(---) ✓
ret;
to (a)

$x \rightarrow R = \text{SA}(x, m+1, \text{ord});$

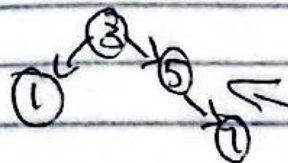
(b) SAC(---)
if(---) ✗
int mid = 1; mid = 4

new nod:

$6 \rightarrow L = 7$



(a) SAC(---)
if(---) ✗
int m = 2; mid = 2
nod new;
↓
continue.



(Question # 07)

// same as Q # 4 of BST